

Lab3-assignment-sentiment

May 4, 2026

1 Lab3 - Assignment Sentiment

Copyright: Vrije Universiteit Amsterdam, Faculty of Humanities, CLTL

This notebook describes the LAB-3 assignment of the Text Mining course. It is about sentiment analysis.

The aims of the assignment are: * Learn how to run a rule-based sentiment analysis module (VADER) * Learn how to run a machine learning sentiment analysis module (Scikit-Learn/ Naive Bayes) * Learn how to run scikit-learn metrics for the quantitative evaluation * Learn how to perform and interpret a quantitative evaluation of the outcomes of the tools (in terms of Precision, Recall, and F1) * Learn how to evaluate the results qualitatively (by examining the data) * Get insight into differences between the two applied methods * Get insight into the effects of using linguistic preprocessing * Be able to describe differences between the two methods in terms of their results * Get insight into issues when applying these methods across different domains

In this assignment, you are going to create your own gold standard set from 50 tweets. You will use the VADER and scikit-learn classifiers on these tweets and evaluate the results by using evaluation metrics and inspecting the data.

We recommend you go through the notebooks in the following order: * **Read the assignment (see below)** * **Lab3.2-Sentiment-analysis-with-VADER.ipynb** * **Lab3.3-Sentiment-analysis-with-scikit-learn.ipynb** * **Answer the questions of the assignment (see below) using the provided notebooks and submit**

In this assignment you are asked to perform both quantitative evaluations and error analyses: * a quantitative evaluation concerns the scores (Precision, Recall, and F1) provided by scikit's `classification_report`. It includes the scores per category, as well as micro and macro averages. Discuss whether the scores are balanced or not between the different categories (positive, negative, neutral) and between precision and recall. Discuss the shortcomings (if any) of the classifier based on these scores * an error analysis regarding the misclassifications of the classifier. It involves going through the texts and trying to understand what has gone wrong. It serves to get insight in what could be done to improve the performance of the classifier. Do you observe patterns in misclassifications? Discuss why these errors are made and propose ways to solve them.

1.1 Credits

The notebooks in this block have been originally created by [Marten Postma](#) and [Isa Maks](#). Adaptations were made by [Filip Ilievski](#).

1.2 Part I: VADER assignments

1.2.1 Preparation (nothing to submit):

To be able to answer the VADER questions you need to know how the tool works. * Read more about the VADER tool in [this blog](#).

* VADER provides 4 scores (positive, negative, neutral, compound). Be sure to understand what they mean and how they are calculated. * VADER uses rules to handle linguistic phenomena such as negation and intensification. Be sure to understand which rules are used, how they work, and why they are important. * VADER makes use of a sentiment lexicon. Have a look at the lexicon. Be sure to understand which information can be found there (lemma?, wordform?, part-of-speech?, polarity value?, word meaning?) What do all scores mean? https://github.com/cjhutto/vaderSentiment/blob/master/vaderSentiment/vader_lexicon.txt

1.2.2 [3.5 points] Question1:

Regard the following sentences and their output as given by VADER. Regard sentences 1 to 7, and explain the outcome **for each sentence**. Take into account both the rules applied by VADER and the lexicon that is used. You will find that some of the results are reasonable, but others are not. Explain what is going wrong or not when correct and incorrect results are produced.

INPUT SENTENCE 1 I love apples

VADER OUTPUT {'neg': 0.0, 'neu': 0.192, 'pos': 0.808, 'compound': 0.6369}

INPUT SENTENCE 2 I don't love apples

VADER OUTPUT {'neg': 0.627, 'neu': 0.373, 'pos': 0.0, 'compound': -0.5216}

INPUT SENTENCE 3 I love apples :-)

VADER OUTPUT {'neg': 0.0, 'neu': 0.133, 'pos': 0.867, 'compound': 0.7579}

INPUT SENTENCE 4 These houses are ruins

VADER OUTPUT {'neg': 0.492, 'neu': 0.508, 'pos': 0.0, 'compound': -0.4404}

INPUT SENTENCE 5 These houses are certainly not considered ruins

VADER OUTPUT {'neg': 0.0, 'neu': 0.51, 'pos': 0.49, 'compound': 0.5867}

INPUT SENTENCE 6 He lies in the chair in the garden

VADER OUTPUT {'neg': 0.286, 'neu': 0.714, 'pos': 0.0, 'compound': -0.4215}

INPUT SENTENCE 7 This house is like any house

VADER OUTPUT {'neg': 0.0, 'neu': 0.667, 'pos': 0.333, 'compound': 0.3612}

1.2.3 [Points: 2.5] Exercise 2: Collecting 50 tweets for evaluation

Collect 50 tweets. Try to find tweets that are interesting for sentiment analysis, e.g., very positive, neutral, and negative tweets. These could be your own tweets (typed in) or collected from the Twitter stream. If you have trouble accessing Twitter, try to find an existing dataset (on websites like kaggle or huggingface).

We will store the tweets in the file **my_tweets.json** (use a text editor to edit). For each tweet, you

should insert: * sentiment analysis label: negative | neutral | positive (this you determine yourself, this is not done by a computer) * the text of the tweet * the Tweet-URL

from:

```
"1": {
    "sentiment_label": "",
    "text_of_tweet": "",
    "tweet_url": "",
```

to:

```
"1": {
    "sentiment_label": "positive",
    "text_of_tweet": "All across America people chose to get involved, get engaged and star",
    "tweet_url" : "https://twitter.com/BarackObama/status/946775615893655552",
},
```

You can load your tweets with human annotation in the following way.

```
[13]: import json
```

```
[14]: my_tweets = json.load(open('my_tweets.json'))
```

```
[15]: for id_, tweet_info in my_tweets.items():
        print(id_, tweet_info)
        break
```

```
1 {'sentiment_label': 'negative', 'text_of_tweet': 'THIS EXACTLY, I'm tired of
seeing so many negative Nancie's acting like we don't have more picks in the
draft, like get your panties out of the twist they're in', 'tweet_url':
'https://x.com/wolverinequeen0/status/2047670187038511517'}
```

1.2.4 [5 points] Question 3:

Run VADER on your own tweets (see function `run_vader` from notebook **Lab3.2-Sentiment-analysis-using-VADER.ipynb**). You can use the code snippet below this explanation as a starting point. * [2.5 points] a. Perform a quantitative evaluation. Explain the different scores, and explain which scores are most relevant and why. * [2.5 points] b. Perform an error analysis: select 10 positive, 10 negative and 10 neutral tweets that are not correctly classified and try to understand why. Refer to the VADER-rules and the VADER-lexicon. Of course, if there are less than 10 errors for a category, you only have to check those. For example, if there are only 5 errors for positive tweets, you just describe those.

```
[16]: def vader_output_to_label(vader_output):
        """
        map vader output e.g.,
        {'neg': 0.0, 'neu': 0.0, 'pos': 1.0, 'compound': 0.4215}
        to one of the following values:
        a) positive float -> 'positive'
        b) 0.0 -> 'neutral'
```

```

c) negative float -> 'negative'

:param dict vader_output: output dict from vader

:rtype: str
:return: 'negative' / 'neutral' / 'positive'
"""
compound = vader_output['compound']

if compound < 0:
    return 'negative'
elif compound == 0.0:
    return 'neutral'
elif compound > 0.0:
    return 'positive'

assert vader_output_to_label( {'neg': 0.0, 'neu': 0.0, 'pos': 1.0, 'compound': 0.0}) == 'neutral'
assert vader_output_to_label( {'neg': 0.0, 'neu': 0.0, 'pos': 1.0, 'compound': 0.01}) == 'positive'
assert vader_output_to_label( {'neg': 0.0, 'neu': 0.0, 'pos': 1.0, 'compound': -0.01}) == 'negative'

```

```

[17]: from sklearn.metrics import classification_report
from nltk.sentiment.vader import SentimentIntensityAnalyzer
vader_model = SentimentIntensityAnalyzer()

tweets = []
all_vader_output = []
gold = []

# settings (to change for different experiments)
to_lemmatize = True
pos = set()

for id_, tweet_info in my_tweets.items():
    the_tweet = tweet_info['text_of_tweet']
    vader_output = vader_model.polarity_scores(the_tweet) # '# run vader
    vader_label = vader_output_to_label(vader_output) # '# convert vader output
    to category

    tweets.append(the_tweet)
    all_vader_output.append(vader_label)
    gold.append(tweet_info['sentiment_label'])

# use scikit-learn's classification report

```

```
report = classification_report(gold, all_vader_output)
print(report)
```

	precision	recall	f1-score	support
negative	0.71	0.77	0.74	22
neutral	0.67	0.27	0.38	15
positive	0.50	0.77	0.61	13
accuracy			0.62	50
macro avg	0.62	0.60	0.58	50
weighted avg	0.64	0.62	0.60	50

```
[18]: error_lis = []

for tweet, gold_label, prediction in zip(tweets, gold, all_vader_output):
    if (gold_label != prediction):
        error_lis.append((tweet, gold_label, prediction))

positive_errs = []
neutral_errs = []
negative_errs = []

for er in error_lis:
    if(er[1] == 'positive'):
        positive_errs.append(er)
    elif(er[1] == 'neutral'):
        neutral_errs.append(er)
    elif(er[1] == 'negative'):
        negative_errs.append(er)

print(f'Positive Errors: {positive_errs[:10]}')
```

Positive Errors: [('i'm so glad i went into project hail mary blind i hadn't even seen a trailer or read a plot synopsis i was just bored as fuck and wanted to go to the movies and that shit changed my life', 'positive', 'negative'), ('Nyc has taught me that if you only just ask you will receive', 'positive', 'neutral'), ('Previous scholars have argued that it is so over. In this paper, however, I posit that we are so back', 'positive', 'negative')]

```
[19]: print(f'Neutral Errors: {neutral_errs[:10]}')
```

Neutral Errors: [('If the person you're duoding with has to get on an alt to duo with you, it's boosting', 'neutral', 'positive'), ('Trump announces he is considering expelling Spain from NATO. NATO has to remind him he has no authority to expel any nation.', 'neutral', 'negative'), ('My feed is full of

posts about scientists being killed over the last few weeks because they're connected to nuclear fusion. Is there any legitimate reporting on this? Or did my algorithm just find an odd section of Twitter?', 'neutral', 'negative'), ('california is the only state where you can take a real vacation without having to leave the state', 'neutral', 'positive'), ('hungry jacks is better than maccas and its not even close bro sorry', 'neutral', 'positive'), ('First time in Norway. Everyone speaks norwegian here. Cool.', 'neutral', 'positive'), ('Jynxzi announces he'll be playing League of Legends for the very first time this Sunday coached by Tyler1, in an attempt to climb out of Iron the lowest rank people said I'm going to quit the game in an hour.. If I apply every part of my braincells I can conquer that game", 'neutral', 'negative'), ('What if I told you there is physics behind luck', 'neutral', 'positive'), ('I care about my grades but if I'm tired I'm going to sleep idc", 'neutral', 'negative'), ('Canadians are like I'm gonna go use the washroom', 'neutral', 'positive')]

```
[20]: print(f'Negative Errors: {negative_errs[:10]}')
```

Negative Errors: [('there is a kid at a neighbouring school called sid seven. wish him the best, last year mustve been hell', 'negative', 'positive'), ('if you watch streamers you're subhuman to me', 'negative', 'neutral'), ("Why is there no competitor to LinkedIn? - It's slow - It's full of AI slop - It actively encourages people to write AI slop- It's slow - The timeline is awful - It's slow There is clearly a market for a professional social network. I would jump in a second.", 'negative', 'positive'), ("How the fuck am I supposed to get a job if they ask for at least 1-2 years of work experience. I'M FRESH OUT OF COLLEGE how am I supposed to have experience if they won't give me a damn chance", 'negative', 'positive'), ("I'd rather drink the water from a tin of tuna than be asked to share a fun fact about myself as an icebreaker during a team building activity at work.", 'negative', 'positive')]

Question 3 Written Portion

Precision scores show how many of the positive predictions are trully positive

Recall scores show what percent of positive labels did you find

f1-scores shows a balanced metric of both precision and recall

Support shows how many instances there are of a class label

—

3a) In the case of our sentiment analysis, we believe that both predicting correctly and finding all correct class labels is important. Therefore we bleieve taht the most important meytric for us is the f1-score.

3b)

For the tweets which are *positive*, there are three insances where a tweet is misclassified as negative or neutral instead. The first one has the text “iâ€™m so glad i went into project hail mary blind i hadnâ€™t even seen a trailer or read a plot synopsis i was just bored as fuck and wanted to go to the movies and that shit changed my life”. This tweet has a positive sentiment because the movie

changed the person's life. However, it likely recieved a negative label prediction because of the vulgar language and the word 'bored' (particularly when the word 'bored' is accenuated by vulgar language) (intesifier and vulgar language). The next tweet is 'Nyc has taught me that if you only just ask you will receive'. This tweet has a positive (hopeful) sentiment, however the model predicts it to be neutral. This is because their is no use of strong language here, and perhaps the positive sentiment is cultural/contextual in nature. The final tweet is 'Previous scholars have argued that it is so over. In this paper, however, I posit that we are so back'. The language of the tweet is a little confusing and relies upon new phrases used online (we are so back, it's so over). It's not clear why the model would predict this tweet as negative, I would think that the model would predict it as neutral. ***Across all these tweets, the actual context of what's being said cannot be understood by the model, which leads to the classification errors we see. This is furthered by intensifiers and specific word choice.***

For the tweets which are *neutral*, we will observe ten instances where a tweet is misclassified as positive or negative instead. The first tweet is: 'Trump announces he is considering expelling Spain from NATO. NATO has to remind him he has no authority to expel any nation.' This tweet is just a factual statement regarding the news, however, because of the presence of the negative word 'expel', this sentence is labeled as negative. The next tweet is, My feed is full of posts about scientists being killed over the last few weeks because theyâ€™re connected to nuclear fusion. Is there any legitimate reporting on this? Or did my algorithm just find an odd section of Twitter?. This is again just a factual statement, but because the topic of the statement is death, it is labeled as negative. The next tweet is: 'If the person you're duoinng with has to get on an alt to duo with you, it's boosting'. This tweet was classified as being positive despite just being a statement of fact regarding streaming. VADER sees the word boosting and thinks that this is a positive sentiment tweet because it lacks the context to understand streaming and what boosting is in streaming. The next tweet is 'california is the only state where you can take a real vacation without having to leave the state', again this is just a calm expression of opinion, yet VADER classifies it as positive despite the contextually neutral sentiment. The next tweet is 'hungry jacks is better than maccas and its not even close bro sorry'. This tweet seems to just be a conversation where someone is saying one resteraunt is better than another. The word better results in a positive sentiment classification by VADER even though it is a statement of opinion in a conversation. The next tweet is 'First time in Norway. Everyone speaks norwegian here. Cool.' This is merely an observation, for the author, cool is a means of acknowledgement. VADER on the other hand interprets this positively. The next tweet is "Jynxzi announces he'll be playing League of Legends for the very first time this Sunday coached by Tyler1, in an attempt to climb out of Iron the lowest rank people said I'm going to quit the game in an hour.. If I apply every part of my braincells I can conquer that game", this tweet is slighly difficult to understand because it is both a statement of an event to occur as well as an opinion. It is this confusion that makes most people view this is as just neutral. VADER however seems to interpret the opinion as criticism, and gives the tweet a negative sentiment rating. The next tweet is 'What if I told you there is physics behind luck', this is ranked as positive despite just being a neutral statement. This is likely beause of the word 'luck'. The next tweet is 'I care about my grades but if I'm tired l'm going to sleep idc'. Once again, this is a statement of fact of how someone plans on acting. VADER seems to pick up the words 'tired' and 'idc' and gives the tweet a negative sentiment rating. The final tweet is: 'Canadians are like Iâ€™m gonna go use the washroom', this tweet is once again just a statement / observation, but the use of like makes the model believe this is positive or a joke. ***Across all these tweets, ambiguous language and factual statements / observations are missclassified because of the underlying topics being discussed as well as the words being used. VADER seems to struggle with this***

as it was surprising to see the tweets which were wrongly classified when their gold label is neutral.

For the tweets which are *negative*, we will observe five instances where a tweet is misclassified as positive or neutral instead. The first tweet is ‘there is a kid at a neighbouring school called sid seven. wish him the best, last year mustve been hell’. This tweet is negative because it is someone who is sad for a kid who will face bullying, however the model believes it to be a positive tweet because the author wishes the kid the best. The next tweet is “How the fuck am I supposed to get a job if they ask for at least 1-2 years of work experience. I’M FRESH OUT OF COLLEGE how am I supposed to have experience if they won’t give me a damn chance”. This is obviously a negative tweet written in anger and desperation because of the author’s difficulty finding a job. The model instead classifies this tweet as positive as it fails to understand the lived experience and desperation of which the author writes. The next tweet is The final tweet is Why is there no competitor to LinkedIn? - It’s slow - It’s full of AI slop - It actively encourages people to write AI slop- It’s slow - The timeline is awful - It’s slow There is clearly a market for a professional social network. I would jump in a second” This tweet expresses negative sentiment about linkedin, however VADER seems to focus in on the portion talking about a potential competitor and how the user would switch given the chance, resulting in a positive ranking of the tweet. The next tweet is “I’d rather drink the water from a tin of tuna than be asked to share a fun fact about myself as an icebreaker during a team building activity at work”. This tweet is obviously negative as the author is sharing extreme discontent with icebreakers at works and even saying they would prefer something else which many readers would understand as negative. However, VADER assigns this tweet a positive rating likely because of words like ‘fun’ and ‘team’. VADER also lacks the necessary context to understand that many people dislike icebreakers and that many people would be disgusted by drinking the water from a tin of tuna. The final tweet is ‘if you watch streamers you’re subhuman to me’, this tweet is obviously negative as it espouses hate towards people. The model fails to understand this and instead labels the tweet as neutral. ***Across all these tweets, context and lived experience result in a situation where readers can understand the true sentiment but the model cannot.***

1.2.5 [4 points] Question 4:

Run VADER on the set of airline tweets with the following settings:

- Run VADER (as it is) on the set of airline tweets
- Run VADER on the set of airline tweets after having lemmatized the text
- Run VADER on the set of airline tweets with only adjectives
- Run VADER on the set of airline tweets with only adjectives and after having lemmatized the text
- Run VADER on the set of airline tweets with only nouns
- Run VADER on the set of airline tweets with only nouns and after having lemmatized the text
- Run VADER on the set of airline tweets with only verbs
- Run VADER on the set of airline tweets with only verbs and after having lemmatized the text

- [1 point] a. Generate for all separate experiments the classification report, i.e., Precision, Recall, and F1 scores per category as well as micro and macro averages. **Use a different code cell (or multiple code cells) for each experiment.**
- [3 points] b. Compare the scores and explain what they tell you.
 - Does lemmatisation help? Explain why or why not.
 - Are all parts of speech equally important for sentiment analysis? Explain why or why not.

```
[21]: import spacy
nlp = spacy.load("en_core_web_sm")

def run_exp(my_tweets, lemmatize = False, parts_of_speech_to_consider = None):

    #doc = nlp(textual_unit)

    preds = []
    gold = []

    for id_, tweet_info in my_tweets.items():

        doc = nlp(tweet_info['text_of_tweet'])
        input_to_vader = []

        for sent in doc.sents:
            for token in sent:

                to_add = token.text

                if lemmatize:
                    to_add = token.lemma_

                if to_add == '-PRON-':
                    to_add = token.text

                if parts_of_speech_to_consider:
                    if token.pos_ in parts_of_speech_to_consider:
                        input_to_vader.append(to_add)
                else:
                    input_to_vader.append(to_add)

        scores = vader_model.polarity_scores(' '.join(input_to_vader))

        label = vader_output_to_label(scores)
        preds.append(label)
        gold.append(tweet_info['sentiment_label'])
```

```
report = classification_report(gold, preds)

return report
```

```
[22]: #loading the airline tweets

import pathlib
import nltk
from nltk.corpus import stopwords
from sklearn.datasets import load_files

# Load the airline tweets
cwd = pathlib.Path.cwd()
airline_tweets_dir = cwd.joinpath('airlinetweets', 'airlinetweets')
airline_tweets_train = load_files(airline_tweets_dir)

airline_tweets = {}
for i in range(len(airline_tweets_train.data)):
    text = airline_tweets_train.data[i].decode("utf-8", errors = "ignore")
    label = airline_tweets_train.target_names[airline_tweets_train.target[i]]

    airline_tweets[i] = {
        "text_of_tweet": text,
        "sentiment_label": label
    }
```

```
[23]: # Run VADER (as it is) on the set of airline tweets
print(run_exp(airline_tweets))
```

	precision	recall	f1-score	support
negative	0.80	0.51	0.63	1750
neutral	0.60	0.51	0.55	1515
positive	0.56	0.88	0.68	1490
accuracy			0.63	4755
macro avg	0.65	0.64	0.62	4755
weighted avg	0.66	0.63	0.62	4755

```
[24]: # Run VADER on the set of airline tweets after having lemmatized the text
print(run_exp(airline_tweets, lemmatize = True))
```

	precision	recall	f1-score	support
negative	0.78	0.52	0.63	1750

neutral	0.60	0.49	0.54	1515
positive	0.56	0.88	0.68	1490
accuracy			0.62	4755
macro avg	0.65	0.63	0.62	4755
weighted avg	0.65	0.62	0.62	4755

```
[25]: # Run VADER on the set of airline tweets with only adjectives
print(run_exp(airline_tweets, parts_of_speech_to_consider = {"ADJ"}))
```

	precision	recall	f1-score	support
negative	0.86	0.20	0.33	1750
neutral	0.40	0.89	0.55	1515
positive	0.67	0.44	0.53	1490
accuracy			0.50	4755
macro avg	0.64	0.51	0.47	4755
weighted avg	0.65	0.50	0.46	4755

```
[26]: # Run VADER on the set of airline tweets with only adjectives and after having
↳ lemmatized the text
print(run_exp(airline_tweets, lemmatize = True, parts_of_speech_to_consider =
↳ {"ADJ"}))
```

	precision	recall	f1-score	support
negative	0.86	0.20	0.33	1750
neutral	0.40	0.89	0.55	1515
positive	0.67	0.44	0.53	1490
accuracy			0.50	4755
macro avg	0.64	0.51	0.47	4755
weighted avg	0.65	0.50	0.46	4755

```
[27]: # Run VADER on the set of airline tweets with only nouns
print(run_exp(airline_tweets, parts_of_speech_to_consider = {"NOUN"}))
```

	precision	recall	f1-score	support
negative	0.73	0.14	0.23	1750
neutral	0.36	0.82	0.50	1515
positive	0.53	0.35	0.42	1490
accuracy			0.42	4755

macro avg	0.54	0.44	0.39	4755
weighted avg	0.55	0.42	0.38	4755

```
[28]: # Run VADER on the set of airline tweets with only nouns and after having
      ↪ lemmatized the text
print(run_exp(airline_tweets, lemmatize=True,
      ↪ parts_of_speech_to_consider={"NOUN"}))
```

	precision	recall	f1-score	support
negative	0.71	0.15	0.25	1750
neutral	0.36	0.81	0.50	1515
positive	0.52	0.34	0.41	1490
accuracy			0.42	4755
macro avg	0.53	0.44	0.39	4755
weighted avg	0.54	0.42	0.38	4755

```
[29]: # Run VADER on the set of airline tweets with only verbs
print(run_exp(airline_tweets, parts_of_speech_to_consider={"VERB"}))
```

	precision	recall	f1-score	support
negative	0.79	0.29	0.42	1750
neutral	0.38	0.81	0.52	1515
positive	0.57	0.35	0.43	1490
accuracy			0.47	4755
macro avg	0.58	0.48	0.46	4755
weighted avg	0.59	0.47	0.46	4755

```
[30]: # Run VADER on the set of airline tweets with only verbs and after having
      ↪ lemmatized the text
print(run_exp(airline_tweets, lemmatize=True,
      ↪ parts_of_speech_to_consider={"VERB"}))
```

	precision	recall	f1-score	support
negative	0.75	0.29	0.42	1750
neutral	0.38	0.78	0.51	1515
positive	0.58	0.36	0.44	1490
accuracy			0.47	4755
macro avg	0.57	0.48	0.46	4755
weighted avg	0.58	0.47	0.45	4755

3B

Compare the scores and explain what they tell you. Does lemmatisation help? Explain why or why not. Are all parts of speech equally important for sentiment analysis? Explain why or why not.

Answer: Across the 8 experiments, it didn't seem that lemmatization had a major effect on the confusion matrix. In some cases, it decreased accuracy, and in others it didn't. Although there would be some changes in the confusion matrix (between lematized and non lematized data), the changes would often be small, and would be offset by increases and decreases in various metrics across the matrix. Comparing the matrices of using only nouns vs using only adjectives, we see that adjectives may be the most important part of speech when it comes to sentiment analysis using VADER. Logically, this makes sense because it is adjectives which describe the quality of different things.

1.3 Part II: scikit-learn assignments

1.3.1 [4 points] Question 5

Train the scikit-learn classifier (Naive Bayes) using the airline tweets.

- Train the model on the airline tweets with 80% training and 20% test set and default settings (TF-IDF representation, min_df=2)
- Train with different settings:
 - with respect to vectorizing: TF-IDF ('airline_tfidf') vs. Bag of words representation ('airline_count')
 - with respect to the frequency threshold (min_df). Carry out experiments with increasing values for document frequency (min_df = 2; min_df = 5; min_df = 10)
- [1 point] a. Generate a classification_report for all experiments
- [3 points] b. Look at the results of the experiments with the different settings and try to explain why they differ:
 - which category performs best, is this the case for any setting?
 - does the frequency threshold affect the scores? Why or why not according to you?

```
[56]: import pathlib
import nltk
from nltk.corpus import stopwords
from sklearn.datasets import load_files

# Load the airline tweets
cwd = pathlib.Path.cwd()
airline_tweets_dir = cwd.joinpath('airlinetweets', 'airlinetweets')
airline_tweets_train = load_files(airline_tweets_dir)

# Validate dataset load
print("Airline tweets dataset statistics")
print("size: ", len(airline_tweets_train.data))
print("categories: ", airline_tweets_train.target_names)
```

```
# Split into test and trains
# airline_train, airline_test = train_test_split(airlines_train)
```

```
Airline tweets dataset statistics
size: 4755
categories: ['negative', 'neutral', 'positive']
```

Count Vectorize data with different min doc frequencies

```
[57]: from sklearn.feature_extraction.text import CountVectorizer

# Create different count
airline_df2_count_vectorizer = CountVectorizer(min_df=2, tokenizer=nlk.
    ↪word_tokenize, stop_words=stopwords.words('english'))
airline_df5_count_vectorizer = CountVectorizer(min_df=5, tokenizer=nlk.
    ↪word_tokenize, stop_words=stopwords.words('english'))
airline_df10_count_vectorizer = CountVectorizer(min_df=10, tokenizer=nlk.
    ↪word_tokenize, stop_words=stopwords.words('english'))

# Fit and Transform using above vectorizers to produce matrices containing
    ↪vectors of word frequency in each document.
airline_count_df2 = airline_df2_count_vectorizer.
    ↪fit_transform(airline_tweets_train.data)
airline_count_df5 = airline_df5_count_vectorizer.
    ↪fit_transform(airline_tweets_train.data)
airline_count_df10 = airline_df10_count_vectorizer.
    ↪fit_transform(airline_tweets_train.data)

# Print vocabulary sizes for each vectorizer
print("Vocab size using occurence in min 2 documents: ",
    ↪len(airline_df2_count_vectorizer.vocabulary_))
print("Vocab size using occurence in min 5 documents: ",
    ↪len(airline_df5_count_vectorizer.vocabulary_))
print("Vocab size using occurence in min 10 documents: ",
    ↪len(airline_df10_count_vectorizer.vocabulary_))
```

```
/Users/adnan/anaconda3/envs/vu-tm-labs/lib/python3.12/site-
packages/sklearn/feature_extraction/text.py:526: UserWarning: The parameter
'token_pattern' will not be used since 'tokenizer' is not None'
    warnings.warn(
/Users/adnan/anaconda3/envs/vu-tm-labs/lib/python3.12/site-
packages/sklearn/feature_extraction/text.py:411: UserWarning: Your stop_words
may be inconsistent with your preprocessing. Tokenizing the stop words generated
tokens ['d', 'll', 'm', 're', 's', 've', 'could', 'might', 'must', 'n't',
'need', 'sha', 'wo', 'would'] not in stop_words.
    warnings.warn(
```

```
Vocab size using occurence in min 2 documents: 2908
```

Vocab size using occurence in min 5 documents: 1232
Vocab size using occurence in min 10 documents: 654

Transform word count matrices into TF_IDF matrices

```
[58]: from sklearn.feature_extraction.text import TfidfTransformer

tfidf_transformer = TfidfTransformer()
airline_tfidf_df2 = tfidf_transformer.fit_transform(airline_count_df2)
airline_tfidf_df5 = tfidf_transformer.fit_transform(airline_count_df5)
airline_tfidf_df10 = tfidf_transformer.fit_transform(airline_count_df10)
```

Train Naive Bayes Classification Models

```
[59]: from sklearn.naive_bayes import MultinomialNB
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report

TEST_RATIO = 0.2

representations = {
    "bag_of_words": {
        2: airline_count_df2,
        5: airline_count_df5,
        10: airline_count_df10,
    },
    "tfidf": {
        2: airline_tfidf_df2,
        5: airline_tfidf_df5,
        10: airline_tfidf_df10,
    }
}

models = {}
results = {}

y = airline_tweets_train.target

for rep_name, df_versions in representations.items():
    models[rep_name] = {}
    results[rep_name] = {}

    for min_df, X_representaion in df_versions.items():
        docs_train, docs_test, y_train, y_test = train_test_split(
            X_representaion,
            y,
```

```

        test_size=TEST_RATIO,
    )

    clf = MultinomialNB().fit(docs_train, y_train)
    y_pred = clf.predict(docs_test)

    models[rep_name][min_df] = clf

    results[rep_name][min_df] = classification_report(
        y_test,
        y_pred,
        target_names=airline_tweets_train.target_names,
        digits=3
    )

```

Part a

```

[60]: # Print classification results
print(f"Classification results")
for rep_name, df_results in results.items():
    print(f"REPRESENTAION USED: {rep_name}")
    for min_df, classification in df_results.items():
        print(f"min doc frequency setting: {min_df}")
        print(classification)

```

Classification results

REPRESENTAION USED: bag_of_words

min doc frequency setting: 2

	precision	recall	f1-score	support
negative	0.812	0.909	0.858	342
neutral	0.847	0.700	0.766	300
positive	0.841	0.871	0.855	309
accuracy			0.831	951
macro avg	0.833	0.827	0.827	951
weighted avg	0.832	0.831	0.828	951

min doc frequency setting: 5

	precision	recall	f1-score	support
negative	0.817	0.909	0.861	350
neutral	0.840	0.694	0.760	310
positive	0.827	0.869	0.848	291
accuracy			0.826	951
macro avg	0.828	0.824	0.823	951

weighted avg	0.828	0.826	0.824	951
--------------	-------	-------	-------	-----

min doc frequency setting: 10

	precision	recall	f1-score	support
negative	0.818	0.897	0.856	341
neutral	0.794	0.763	0.778	299
positive	0.855	0.797	0.825	311
accuracy			0.822	951
macro avg	0.823	0.819	0.820	951
weighted avg	0.823	0.822	0.821	951

REPRESENTAION USED: tfidf

min doc frequency setting: 2

	precision	recall	f1-score	support
negative	0.818	0.936	0.873	360
neutral	0.861	0.690	0.766	306
positive	0.833	0.860	0.846	285
accuracy			0.834	951
macro avg	0.838	0.828	0.828	951
weighted avg	0.836	0.834	0.831	951

min doc frequency setting: 5

	precision	recall	f1-score	support
negative	0.836	0.896	0.865	357
neutral	0.795	0.746	0.770	280
positive	0.859	0.834	0.847	314
accuracy			0.832	951
macro avg	0.830	0.826	0.827	951
weighted avg	0.831	0.832	0.831	951

min doc frequency setting: 10

	precision	recall	f1-score	support
negative	0.866	0.882	0.874	374
neutral	0.774	0.712	0.741	288
positive	0.777	0.820	0.798	289
accuracy			0.812	951
macro avg	0.806	0.805	0.805	951
weighted avg	0.811	0.812	0.811	951

Part b

1. which category performs best, is this the case for any setting?

Across all settings it seems that the “negative” category performs the best. In both types of representation with varying df settings, the negative beats all the other categories on recall and f1 score.

2. does the frequency threshold affect the scores? Why or why not according to you?

Yes but it does not seem to affect the scores significantly.

In BoW Accuracy, is as follows: $df=2 \rightarrow 0.850$ $df=5 \rightarrow 0.850$ $df=10 \rightarrow 0.828$

In TF_IDF, Accuracy is as follows:

$df=2 \rightarrow 0.811$ $df=5 \rightarrow 0.836$ (best) $df=10 \rightarrow 0.818$

The variation is likely due to smaller values for min doc frequency admitting rare words while also allowing typos in the vocab set. Similarly larger df values removes rarer words and therefore loses signal.

Thats why 5 seems to perform the best

1.3.2 [4 points] Question 6: Inspecting the best scoring features

- Train the scikit-learn classifier (Naive Bayes) model with the following settings (airline tweets 80% training and 20% test; Bag of words representation ('airline_count'), min_df=2)
- [1 point] a. Generate the list of best scoring features per class (see function **important_features_per_class** below) [1 point]
- [3 points] b. Look at the lists and consider the following issues:
 - [1 point] Which features did you expect for each separate class and why?
 - [1 point] Which features did you not expect and why ?
 - [1 point] The list contains all kinds of words such as names of airlines, punctuation, numbers and content words (e.g., ‘delay’ and ‘bad’). Which words would you remove or keep when trying to improve the model and why?

```
[9]: def important_features_per_class(vectorizer, classifier, n=80):
    class_labels = classifier.classes_
    feature_names = vectorizer.get_feature_names_out()
    topn_class1 = sorted(zip(classifier.feature_count_[0],
    ↪ feature_names), reverse=True)[:n]
    topn_class2 = sorted(zip(classifier.feature_count_[1],
    ↪ feature_names), reverse=True)[:n]
    topn_class3 = sorted(zip(classifier.feature_count_[2],
    ↪ feature_names), reverse=True)[:n]
    print("Important words in negative documents")
    for coef, feat in topn_class1:
        print(class_labels[0], coef, feat)
    print("-----")
    print("Important words in neutral documents")
    for coef, feat in topn_class2:
```

```

        print(class_labels[1], coef, feat)
    print("-----")
    print("Important words in positive documents")
    for coef, feat in topn_class3:
        print(class_labels[2], coef, feat)

# example of how to call from notebook:
#important_features_per_class(airline_vec, clf)

```

Question 6:

```

[1]: import pathlib
      from sklearn.datasets import load_files
      from sklearn.naive_bayes import MultinomialNB
      from sklearn.model_selection import train_test_split
      from sklearn.feature_extraction.text import CountVectorizer
      from sklearn.metrics import accuracy_score, classification_report
      from sklearn.pipeline import Pipeline

```

```

[2]: # Load the airline tweets
      cwd = pathlib.Path.cwd()
      airline_tweets_dir = cwd.joinpath('airlinetweets', 'airlinetweets')
      airline_tweets_train = load_files(airline_tweets_dir)

      # Validate dataset load
      print("Airline tweets dataset statistics")
      print("size: ", len(airline_tweets_train.data))
      print("categories: ", airline_tweets_train.target_names)

```

```

Airline tweets dataset statistics
size: 4755
categories: ['negative', 'neutral', 'positive']

```

```

[3]: #Create train and test split
      X = airline_tweets_train.data
      y = airline_tweets_train.target

      x_train, x_test, y_train, y_test = train_test_split(
      X, y, test_size=0.2, random_state=42, stratify=y
      )

```

```

[4]: #Building Classifier and vectorizer pipeline:
      vectorizer = CountVectorizer(min_df=2)
      clf = MultinomialNB()

      airline_count = Pipeline([
          ('vectorizer', vectorizer),

```

```
    ('clf', clf)
])
```

```
[5]: #train model
airline_count.fit(x_train, y_train)
```

```
[5]: Pipeline(steps=[('vectorizer', CountVectorizer(min_df=2)),
                      ('clf', MultinomialNB())])
```

```
[6]: #predict on test set
y_predicted = airline_count.predict(x_test)
```

```
[7]: #Evaluate Model
print("Accuracy:", accuracy_score(y_test, y_predicted), "\n")
print("Classification Report: \n")
print(classification_report(y_test, y_predicted))
```

Accuracy: 0.825446898002103

Classification Report:

	precision	recall	f1-score	support
0	0.84	0.94	0.89	350
1	0.83	0.71	0.76	303
2	0.81	0.82	0.81	298
accuracy			0.83	951
macro avg	0.82	0.82	0.82	951
weighted avg	0.82	0.83	0.82	951

Question 6 Part a:

```
[10]: #Generate import features
important_features_per_class(
    airline_count.named_steps['vectorizer'],
    airline_count.named_steps['clf'],
    n=80
)
```

Important words in negative documents

0 1390.0 united
0 756.0 to
0 511.0 the
0 385.0 you
0 382.0 flight
0 368.0 and

0 335.0 for
0 326.0 on
0 319.0 my
0 275.0 is
0 270.0 in
0 209.0 of
0 203.0 your
0 197.0 that
0 196.0 it
0 170.0 me
0 163.0 not
0 161.0 have
0 152.0 at
0 150.0 no
0 148.0 with
0 143.0 was
0 119.0 this
0 118.0 service
0 117.0 we
0 117.0 can
0 114.0 be
0 106.0 an
0 102.0 virginamerica
0 100.0 from
0 97.0 now
0 94.0 get
0 92.0 customer
0 87.0 are
0 86.0 plane
0 86.0 been
0 85.0 cancelled
0 84.0 they
0 83.0 time
0 83.0 delayed
0 83.0 but
0 83.0 bag
0 79.0 why
0 79.0 just
0 72.0 what
0 72.0 so
0 71.0 co
0 68.0 will
0 68.0 hour
0 67.0 hours
0 66.0 http
0 64.0 up
0 64.0 do
0 63.0 late

0 62.0 out
0 62.0 or
0 62.0 has
0 61.0 still
0 60.0 when
0 60.0 airline
0 59.0 our
0 58.0 gate
0 57.0 how
0 57.0 amp
0 56.0 had
0 56.0 after
0 56.0 about
0 55.0 worst
0 54.0 would
0 53.0 again
0 52.0 help
0 52.0 flights
0 51.0 waiting
0 51.0 like
0 51.0 if
0 49.0 by
0 49.0 as
0 48.0 one
0 47.0 ve
0 47.0 because

Important words in neutral documents

1 609.0 to
1 315.0 jetblue
1 274.0 the
1 267.0 southwestair
1 258.0 united
1 251.0 on
1 240.0 you
1 231.0 flight
1 202.0 for
1 198.0 co
1 187.0 in
1 187.0 americanair
1 184.0 http
1 182.0 my
1 171.0 can
1 166.0 is
1 160.0 usairways
1 141.0 and
1 133.0 from
1 131.0 it

1 120.0 of
1 108.0 have
1 100.0 me
1 92.0 do
1 84.0 with
1 75.0 at
1 75.0 any
1 74.0 get
1 73.0 that
1 72.0 be
1 70.0 what
1 70.0 virginamerica
1 69.0 this
1 68.0 flights
1 68.0 are
1 66.0 we
1 66.0 please
1 65.0 if
1 64.0 will
1 64.0 our
1 58.0 there
1 58.0 need
1 57.0 help
1 55.0 just
1 53.0 how
1 49.0 your
1 48.0 or
1 43.0 so
1 42.0 tomorrow
1 41.0 when
1 41.0 out
1 41.0 fleet
1 41.0 fleek
1 41.0 dm
1 40.0 not
1 40.0 know
1 40.0 an
1 38.0 flying
1 37.0 would
1 37.0 now
1 37.0 am
1 35.0 was
1 35.0 us
1 35.0 thanks
1 35.0 about
1 34.0 no
1 34.0 change
1 33.0 cancelled

1 33.0 as
1 32.0 way
1 31.0 fly
1 30.0 hi
1 29.0 amp
1 28.0 time
1 27.0 see
1 26.0 up
1 26.0 travel
1 26.0 one
1 25.0 number
1 25.0 like

Important words in positive documents

2 460.0 you
2 431.0 the
2 411.0 to
2 340.0 for
2 308.0 southwestair
2 295.0 thanks
2 283.0 jetblue
2 261.0 united
2 248.0 thank
2 208.0 and
2 190.0 flight
2 162.0 americanair
2 156.0 on
2 154.0 my
2 144.0 great
2 137.0 usairways
2 132.0 in
2 125.0 your
2 113.0 it
2 108.0 so
2 106.0 with
2 106.0 is
2 104.0 me
2 97.0 of
2 93.0 service
2 90.0 was
2 89.0 at
2 88.0 virginamerica
2 79.0 this
2 77.0 love
2 73.0 co
2 68.0 http
2 66.0 customer
2 66.0 are

2 65.0 much
2 65.0 best
2 60.0 that
2 60.0 just
2 59.0 guys
2 59.0 from
2 59.0 awesome
2 57.0 be
2 57.0 all
2 55.0 have
2 51.0 we
2 50.0 airline
2 48.0 time
2 47.0 got
2 44.0 up
2 42.0 today
2 42.0 good
2 41.0 us
2 41.0 amazing
2 40.0 will
2 40.0 crew
2 39.0 get
2 39.0 amp
2 37.0 very
2 37.0 out
2 36.0 help
2 35.0 flying
2 35.0 can
2 34.0 made
2 33.0 home
2 33.0 fly
2 32.0 they
2 32.0 our
2 31.0 gate
2 30.0 re
2 30.0 but
2 30.0 an
2 29.0 back
2 28.0 work
2 28.0 now
2 28.0 not
2 28.0 again
2 27.0 response
2 27.0 do
2 26.0 ll
2 25.0 tonight

Question 6 Part b:

Which features did you expect for each separate class and why?

Negative Class:

For the negative class I expected words related to time and delays as I imagined this to be the most complained about frustration for airlines. The words I saw confirmed this. For example, the negative list contains delayed, cancelled, late, hours, etc...

Nuetral Class:

For the nuetral class I expected to see words belonging to generic questions or statements about airlines. This would include airline names and normal flight statement/question terms. I also expected the absence of words containing any sort of positive or negative sentiment such as good or bad. The words I saw confrimed this. For example, the nuetral list contains jetblue, southwestair, help, please, etc...

Positive Class:

For the positive class I expected to see particicularly positive words from customers declaring their gratitude to the airline for a good flight as well as commentary on the quality of the airline. The words I saw confirmed this. For example, the positive list contains thanks, great, love, best, etc..

Which features did you not expect and why ?

In the negative class the word “like” shows up. I did not expect to see this as it generally has a positive connotation. The word “thanks” shows up in the nuetral connotation class. I did not expect to see this as “thanks” also normally has a positive connotation.

The list contains all kinds of words such as names of airlines, punctuation, numbers and content words (e.g., ‘delay’ and ‘bad’). Which words would you remove or keep when trying to improve the model and why?

When trying to imporove the model I would primarily remove all words that show up often in all three classes. This include the names of airlines, words like “and” and “is”, and punctuation. I would remove these because if they show up often in all three classes they cannot be very consequential in helping to differentiate between the classes. This can be seen in the words themselves.

1.3.3 [Optional! (will not be graded)] Question 7

Train the model on airline tweets and test it on your own set of tweets + Train the model with the following settings (airline tweets 80% training and 20% test; Bag of words representation (‘airline_count’), min_df=2) + Apply the model on your own set of tweets and generate the classification report * [1 point] a. Carry out a quantitative analysis. * [1 point] b. Carry out an error analysis on 10 correctly and 10 incorrectly classified tweets and discuss them * [2 points] c. Compare the results (cf. classification report) with the results obtained by VADER on the same tweets and discuss the differences.

1.3.4 [Optional! (will not be graded)] Question 8: trying to improve the model

- [2 points] a. Think of some ways to improve the scikit-learn Naive Bayes model by playing with the settings or applying linguistic preprocessing (e.g., by filtering on part-of-speech, or removing punctuation). Do not change the classifier but continue using the Naive Bayes classifier. Explain what the effects might be of these other settings

- [1 point] b. Apply the model with at least one new setting (train on the airline tweets using 80% training, 20% test) and generate the scores
- [1 point] c. Discuss whether the model achieved what you expected.

1.4 End of this notebook

[]: